

Concurrency Controls, Locking Strategies and Deadlock Management

Topics covered :

- > Race Conditions
- > Isolation Levels
- > Dirty Reads
- > Non-Repeatable Reads
- > Phantom Reads
- > Locking Strategies
- > Deadlock Management

Race Conditions

Dirty Read

A Dirty read is a situation when a transaction reads data that has not yet been committed.

Let's say transaction 1 updates a row and leaves it uncommitted, meanwhile, Transaction 2 reads the updated row. If transaction 1 rolls back the change, transaction 2 will have read data that is considered never to have existed.

Non Repeatable Read

Non-repeatable read occurs when a transaction reads the same row twice and gets a different value each time.

Example:

suppose transaction T1 reads data. Due to concurrency, another transaction T2 updates the same data and commit, Now if transaction T1 rereads the same data, it will retrieve a different value.

Phantom Read

Phantom Read occurs when two same queries are executed, but the rows retrieved by the two, are different.

suppose transaction T1 retrieves a set of rows that satisfy some search criteria. Now, Transaction T2 generates some new rows that match the search criteria for Transaction T1. If transaction T1 re-executes the statement that reads the rows, it gets a different set of rows this time.

1. Read Uncommitted

This is the lowest level of isolation where a transaction can see uncommitted changes made by other transactions. This can result in dirty reads, non-repeatable reads, and phantom reads.

2. Read Committed

In this isolation level, a transaction can only see changes made by other committed transactions. This eliminates dirty reads but can still result in non-repeatable reads and phantom reads.

3. Repeatable Read

This isolation level guarantees that a transaction will see the same data throughout its duration, even if other transactions commit changes to the data. However, phantom reads are still possible.

4. Serializable

This is the highest isolation level where a transaction is executed as if it were the only transaction in the system. All transactions must be executed sequentially, which ensures that there are no dirty reads, non-repeatable reads, or phantom reads.

✗ → May occur

✓ → Don't occur

Isolation Level	Dirty Reads	Non-Repeatable Reads	Phantom Reads
Read Uncommitted	✗	✗	✗
Read Committed	✓	✗	✗
Repeatable Read	✓	✓	✗
Serializable	✓	✓	✓

The isolation level should match the application's needs balancing data accuracy and performance. Higher levels (e.g., Serializable): Strong consistency, lower concurrency, slower performance. Lower levels (e.g., Read Uncommitted): Better concurrency, faster performance, risk of inconsistencies.

Multi-Version Concurrency Control

Multi-Version Concurrency Control (MVCC) is the underlying engine that powers PostgreSQL. The best part? You don't have to build it yourself—PostgreSQL provides it automatically.

In traditional databases, if Request A is updating a row, Request B has to wait just to read it. MVCC changes the game with a golden rule: **Writers never block readers, and readers never block writers.**

Instead of overwriting data in place, PostgreSQL treats rows as immutable. When you update a row, PostgreSQL actually creates a new version of that row alongside the old one.

Every transaction is assigned a unique, incrementing Transaction ID (TxID). PostgreSQL tracks which transaction created or deleted a row version using hidden system columns:

xmin: The TxID of the transaction that inserted the row version.

xmax: The TxID of the transaction that updated or deleted it.

Therefore, we should apply MVCC when we have Optimistic length of transactions, where we know that exactly how many concurrent transactions can be placed at a time. This will be basically based on the two key factors :

-> how many requests fight for the same row

-> how long the transaction takes

Note : Take reference from the [database Engineering/concurrency locking deadlock/routes](#) section and look for the route : [api/v1/withdrawals/withdraw-mvcc](#) for the implementation and how does in beneficial for use cases where we have limited concurrency.

Locking Strategies

Pessimistic Locking

Pessimistic locking operates on a simple assumption: "Conflicts are highly likely, so I will lock the resource immediately before anyone else can touch it." Instead of checking for conflicts at the end of a transaction (like we did with MVCC/Optimistic isolation levels), pessimistic locking halts competing transactions at the beginning. In a relational database like PostgreSQL, this is achieved using row-level locks via the **SELECT ... FOR UPDATE** modifier.

The Execution Mechanics

When a transaction executes a query ending in FOR UPDATE, PostgreSQL attempts to acquire an exclusive lock on the matched rows.

Transaction A requests the row with FOR UPDATE. It acquires the lock and continues executing its business logic.

Transaction B tries to request the same row with FOR UPDATE. PostgreSQL detects the existing lock and forces Transaction B to pause at that exact line of code.

Transaction B's connection remains open and waiting until Transaction A either commits or rolls back.

The moment Transaction A finishes, its lock is released. Transaction B instantly wakes up, reads the freshly updated data left by Transaction A, and proceeds safely.

USECASE : We never use the locking strategies where we have to deal large amounts of users. The locking is best suited for cases where single user can perform two concurrent tasks for its own resource. Like updating balance from the two different platforms at the same time.

Example : Let's consider scenario where multiple Credit/Debit cards are assigned for the single account. One for spouse, for child, for family etc. Now if one transaction happens at shopping mall and second at the restaurant at the same time we will get the race conditions for accessing the same row for the account.

Optimistic Locking

Optimistic locking does not lock the row during reading. Instead, it adds a version or timestamp column to the table. When updating, it checks if the version has changed since it was read. If it has, the update fails.

Best for: Low-concurrency environments where conflicts are rare.

How it works: SQLAlchemy handles this automatically via a `version_id_col`.

There are other locking strategies as well but hopes these gives you the essence of how it works rest you can according to use cases.

Deadlock Management

1. Deadlock Prevention : The DBMS anticipates deadlocks and aborts or delays a transaction before a deadlock occurs. It evaluates transactions using timestamp priorities.**Wait-Die Scheme: Non-preemptive**. If an older transaction requests a resource held by a younger one, it is allowed to wait. If a younger transaction requests a resource from an older one, the younger one dies (aborts).**Wound-Wait Scheme: Preemptive**. If an older transaction requests a resource from a younger one, the older one preempts (wounds) the younger one, forcing it to abort. If a younger transaction requests from an older one, the younger one waits.

2. Deadlock Avoidance : Transactions are required to lock all their resources simultaneously rather than incrementally. If all resources are not available, the transaction waits and acquires nothing, **eliminating the "hold and wait" condition completely**.

3. Deadlock Detection & Recovery : The system regularly runs an algorithm to check for cycles and resolves the deadlock if one is found.

Wait-For Graph (WFG): The DBMS constructs a directed graph where nodes are transactions and edges represent dependencies. **If the graph contains a cycle, a deadlock is present**.

Timeouts: A transaction that waits longer than a specified time limit is assumed to be deadlocked and is aborted.

Recovery: Once a deadlock is detected, the DBMS selects a "victim" transaction—usually based on its age or how few resources it has used—and aborts it. This frees the locks and allows the other transactions to proceed.